GATE
CS & DA

Artificial Intelligence

Un- Informed Search

Lecture No. 06

Recap of Previous Lecture

- Topic (Un Informed Search)
- Topic
- Topic
- Topic
- Topic

Topics to be Covered

- Topic — Informed Search
- Topic — GBFS ⇒ Greedy Best first search
- Topic — A* algo.
- Topic
- Topic

Uninformed search → Not efficient

→ bcoz it goes and Search all path.

Informed search

→ Informed search algorithms use additional information, usually in the form of heuristics, to make more informed decisions about which path to explore in a search space.

→ These algorithms are more efficient than uninformed search strategies (like BFS and DFS) because they use problem-specific knowledge to guide the search process towards the goal more quickly.

Informed search

h(n) ⇒ value of heuristics at nth node

Heuristic Function (h(n)):

- A heuristic is a function that estimates the cost of the cheapest path from a node n to the goal.
- The quality of the heuristic greatly influences the efficiency of the search. A good heuristic should be admissible and consistent.

GBFS ⇒

greedy Best first Search

- in graph we will have weights of each edge
- heuristic value for each node

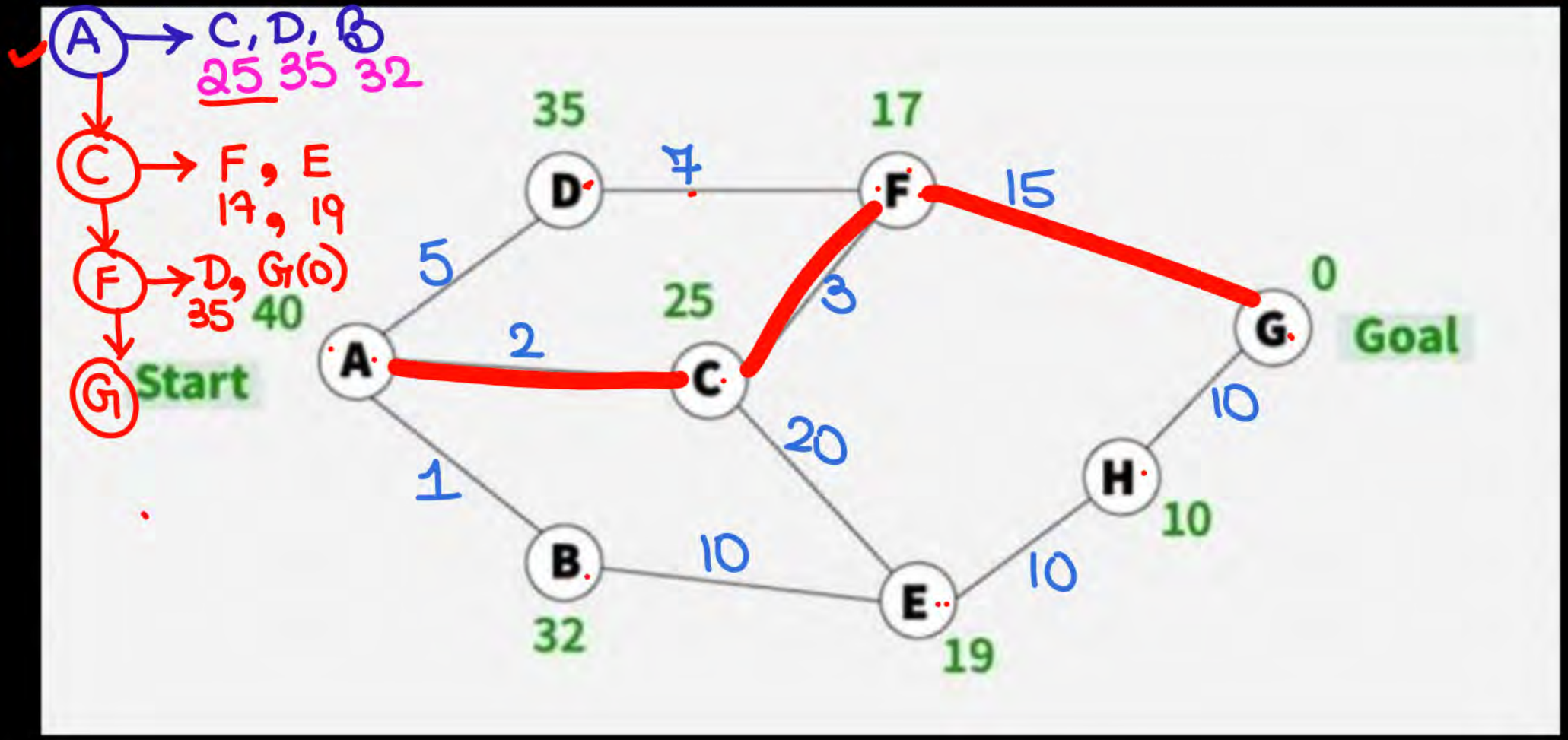
Step1 Start from Start node

Step2 Now Check the neighbour of the present node.

Now from present node move to that neighbour which has min heuristic value

Step3 Repeat step2 until goal node is Reached.

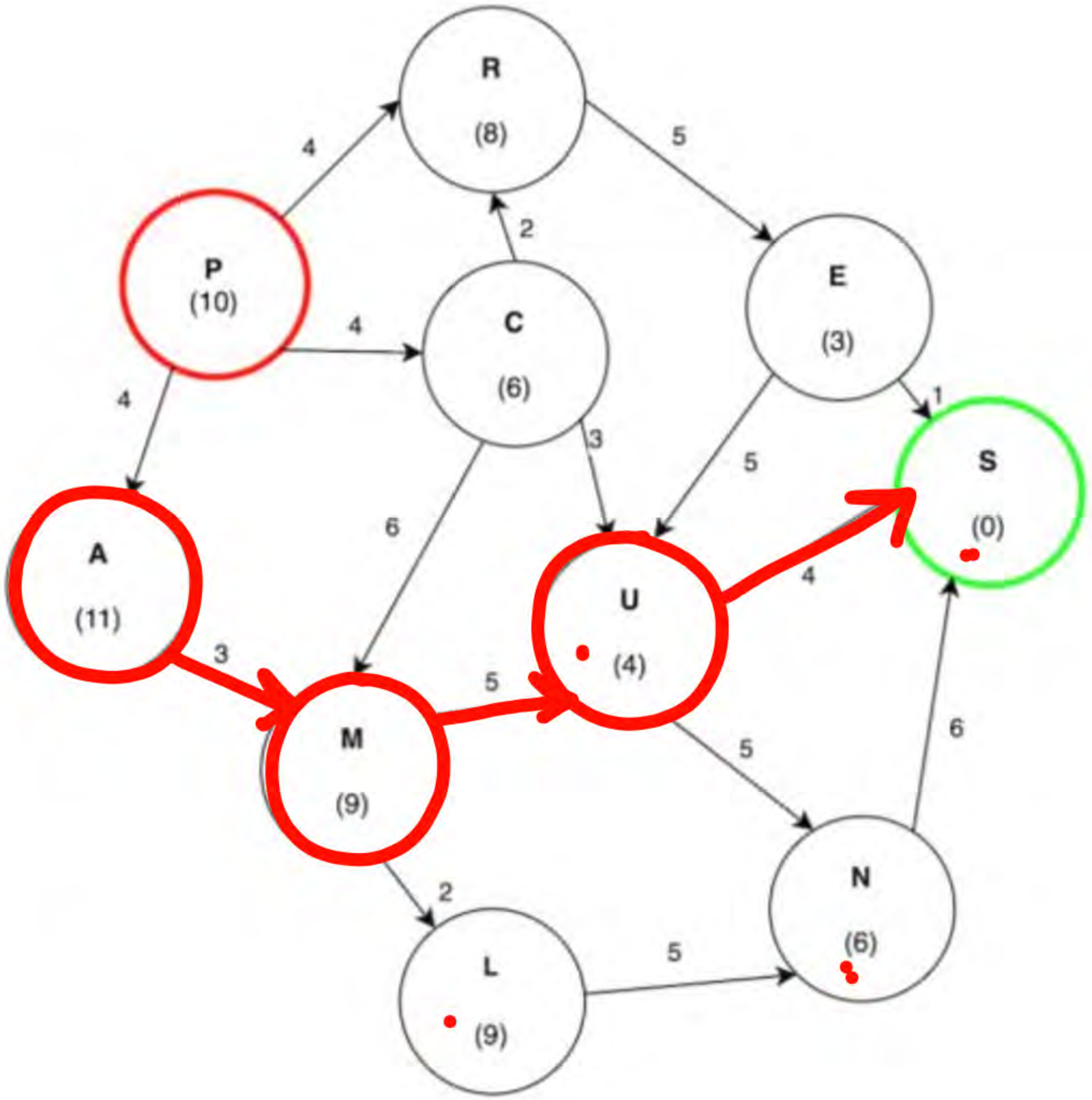
Example


A → C, D, B
25 35 32
C → F, E
17, 19
F → D, G(0)
35 40
G Start
35
17
D
7
F
15
5
0
25
3
G
Goal
A
2
C
10
20
1
H
10
B
10
10
E
32
19

Artificial Intelligence

- Start A → S

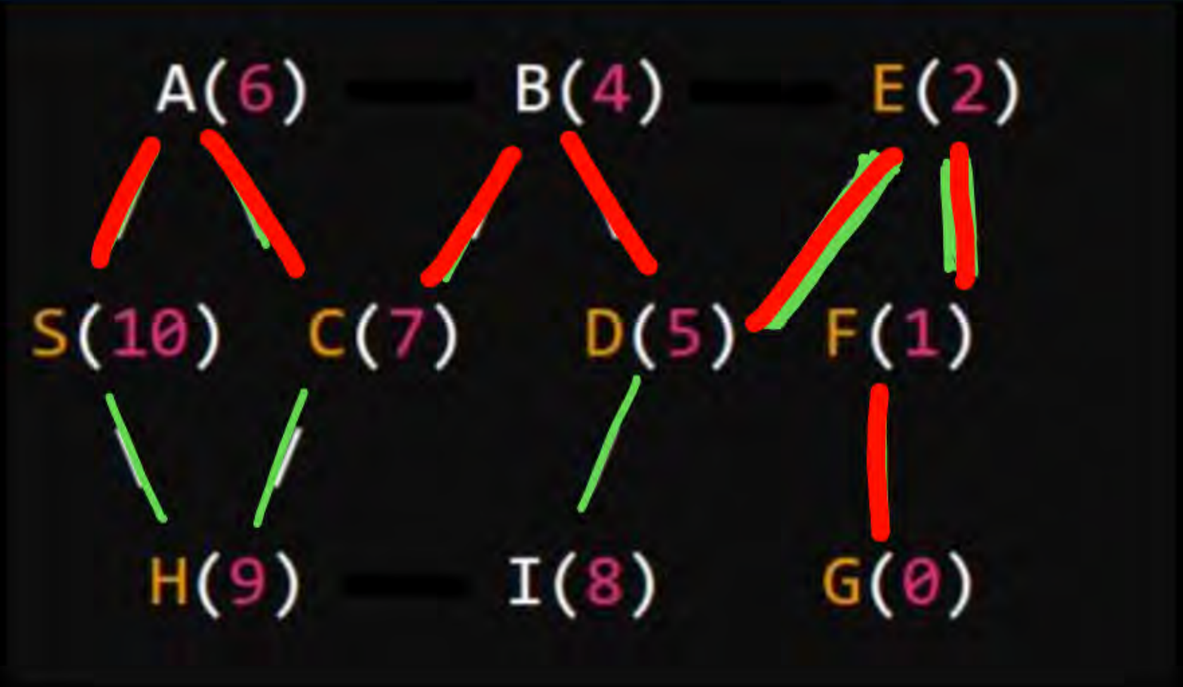
- AMUS :



Example

S → G

S → A, H
6, 9
↓
A → C
↓
C → H, B
9 4
↓
B
↓
D → I, E → E → F → G



Informed search : Best First Search

Key Concepts:

- Informed Search: Search algorithms that use problem-specific knowledge (heuristics) to make decisions.
- Heuristic: A function that estimates the cost from the current node to the goal.

GBFS

- Completeness ✓
- If goal node exist then we will surely Reach goal node

- optimal
- not optimal as, if the heuristic fxn is not efficient then Result the result may have more overall cost

Informed search : Best First Search

- GBFS Overview:
- Search Strategy: GBFS selects the node that seems closest to the goal based on the heuristic function.
- Evaluation Function: f(n)=h(n), where h(n) is the heuristic estimate of the cost to reach the goal from node ~~nn~~(n)
- Non-Optimal: GBFS is not guaranteed to find the optimal solution because it does not consider the cost to reach a node (i.e., g(n)).

Informed search : Best First Search

Algorithm Steps:

- Initialize: Start with the initial node
- Expand: Select the node with the lowest h(n).
- Goal Test: If the node is the goal, stop.
- Expand Neighbours: Add neighbours to the priority queue based on h(n).
- Repeat: Continue until the goal is found or all nodes are explored.

Advantage

Simple Process

Disadvantage

non optimal

heavily dependent on heuristic value

Informed search : Best First Search

Advantages and Disadvantages of GBFS

Advantages:

- Efficiency: Faster than uninformed search algorithms when the heuristic is good.
- Simplicity: Easy to implement and understand.

Disadvantages:

- Suboptimal Solutions: Can lead to suboptimal paths as it ignores the actual cost to reach nodes.
- Stuck in Local Minima: Can get stuck in local minima or loops if the heuristic is misleading.

- Worst Case in GBFS

$TC = O(B^D)$

$SC = O(B^D)$

Bcoz in worst case scenario algo need to go to all nodes.

Informed search : Best First Search

The time and space complexity of Greedy Best-First Search (GBFS) depend on various factors, including the structure of the graph and the heuristic function used. Here's a detailed breakdown:

Time Complexity

1. **Worst-case Time Complexity**:
   - In the worst case, GBFS explores all nodes in the search space until it finds the goal or exhausts all possibilities. The time complexity can be exponential in the number of nodes, especially if the heuristic does not significantly reduce the search space.
   - If the graph has **n** nodes, and the heuristic does not provide significant guidance, GBFS might explore all nodes before finding the goal. Thus, the time complexity can be **O(b^d)**, where **b** is the branching factor and **d** is the depth of the solution.

Informed search : Best First Search

Space Complexity

1. **Space Complexity**:
   - GBFS keeps track of the frontier (the set of nodes to be explored) and the nodes already visited. In the worst case, it may store all nodes in memory if the search does not find the goal or if the goal is at a great depth.
   - The space complexity is therefore also **O(b^d)**, where **b** is the branching factor and **d** is the depth of the deepest node in the search tree. This accounts for storing all nodes in the frontier and possibly all nodes that have been expanded.

Summary

- **Time Complexity**: **O(b^d)** in the worst case, where **b** is the branching factor and **d** is the depth of the solution.
- **Space Complexity**: **O(b^d)**, as it requires storage for the frontier and potentially all explored nodes.

```
      A(10) --- B(8) --- C(7)
     /  \      /  \      /  \
S(15)  D(12)  E(9)   F(6)
 |    /   \   |   \   |
G(0) H(14) I(11) J(5) K(3)
 |   |     |    |    |
 L(1) --- M(13) --- N(4)
```

Informed search : A* search Algorithm

Overview

- A Search* is one of the most widely used search algorithms in Artificial Intelligence for pathfinding and graph traversal.
- It combines the strengths of Dijkstra's Algorithm (which finds the shortest path) and Greedy Best-First Search (which uses a heuristic to guide the search).

Informed search : A* search Algorithm

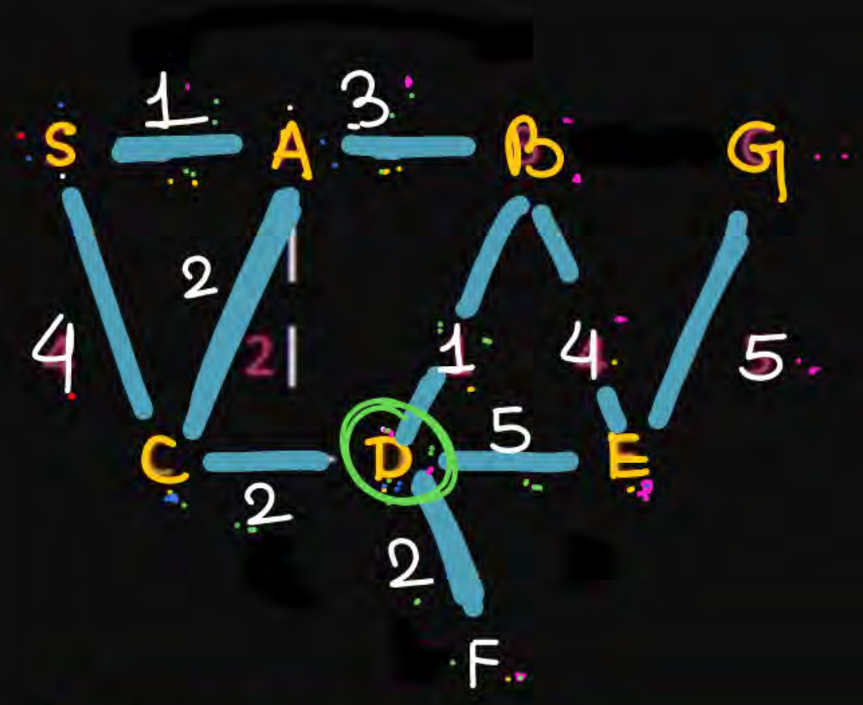
Basic Concepts

Heuristics and Cost Functions

- Heuristic Function (h(n)): An estimate of the cost from node n to the goal.
- Cost Function (g(n)): The actual cost from the start node to node n
- Evaluation Function (f(n)): The sum of the heuristic and cost functions: $f(n)=g(n)+h(n)$

Informed search : A* search Algorithm

- Here we create 2 list Open and closed list…
  - In GBFS ⇒ for deciding where to Jump from present node ⇒ we see h(n)
  - in A* ⇒ " " " " " " " ⇒ " " h(n)+path cost



Heuristic Values (h):

- h(S) = 7
- h(A) = 6
- h(B) = 2
- h(C) = 8
- h(D) = 4
- h(E) = 3
- h(F) = 6
- h(G) = 0

Close list ⇒ S A B D
Open list ⇒ F E C
7+6 =13 11 11

- Start S.
- Open list ⇒ A, C
  (6+1) 7 (4+8) 12.

- Close list ⇒ S, A
- Open list ⇒ B, C
  (2+4) 6 (3+8) =11

- Close list ⇒ S A B
- Open list ⇒ D E, C
  [5+4 9] [8+3 =11] 11

Close list S A B D C

Open list F, E,

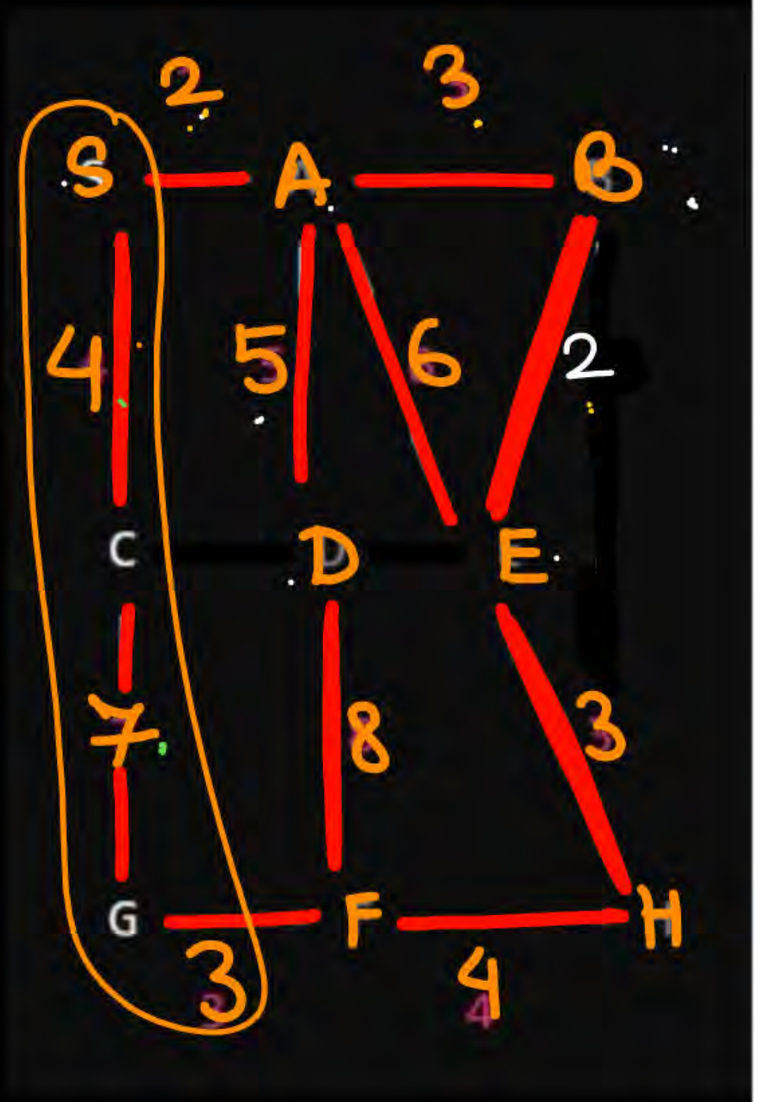
13 11

Close list S A B D C E

open list F, G

13 (13+0)

Close list S A B D C E G



Heuristic Values (h):

- h(S) = 8
- h(A) = 6
- h(B) = 3
- h(C) = 7
- h(D) = 6
- h(E) = 4
- h(F) = 3
- h(G) = 0
- h(H) = 4

UCS + h(n)

	S	A	B	C
S	0	0	0	0
A	8	8	8	8
B	∞	8	8	8
C	11	11	11	11
D	∞	13	13	13
E	∞	12 → 11	11	
F	∞	∞	∞	∞
G	∞	∞	∞	11
H	∞	∞	∞	∞

S ← C ← G

Informed search : A* search Algorithm

A* Search Algorithm

Step-by-Step Process

- Initialize: Place the start node S in the open list with f(S)=g(S)+h(S).
- Expand: Remove the node n with the lowest f(n) from the open list and add it to the closed list.
- Goal Check: If n is the goal node G, the search is complete.
- Generate Successors: For each neighboring node of n:
- Calculate g(n), h(n), and f(n).
- If the node is not in the open or closed list, add it to the open list.
- If it's already in the open list with a higher f(n), update it.
- Repeat: Continue until the goal is found or the open list is empty.

THANK - YOU